



Linear Algebra Project: Entrepreneurial Modeling

Strategic Inventory & Revenue Optimization for a Coffee Shop

Asma Amin	(066)	
Minahil Islam	(001)	
Maniha Riaz	(009)	Leader
Faiza Khalid	(011)	
Rubab Waqar	(020)	

Submitted to: Ms. Sumaira Ajmal

1. Problem Description

In the competitive landscape of the food and beverage industry, an entrepreneur faces the challenge of balancing limited physical resources with aggressive financial targets. "The Daily Grind" must determine the exact production volume for five core product categories to achieve a daily revenue goal of **\$1,420**. The business is constrained by a strict daily ingredient budget, finite staff labor hours, and specific inventory limits for key components like milk. This model seeks to find the "sweet spot" where all these variables intersect perfectly without resulting in waste or missed targets.

2. Purpose / Decision Goal

The primary objective of this project is to apply **Linear Algebra** to entrepreneurial operations by identifying a "Feasible Production Point." By solving the formulated 5 X 5 matrix, the business owner can make data-driven decisions regarding daily inventory.

The Decision Goal is to identify a product mix that satisfies the following:

- **Revenue Achievement:** Reaching a precise daily turnover of **\$1,420** to cover overheads.
- **Budget Compliance:** Staying strictly within a **\$294** procurement budget for raw ingredients.
- **Labor Optimization:** Utilizing **520 minutes** of available staff labor to maximize efficiency without overworking the team.
- **Resource Management:** Fully exhausting the **700 ounces** of milk inventory to ensure freshness and zero waste.
- **Capacity Balancing:** Meeting the **400-unit** total production capacity to ensure the shop is fully stocked but not over-capacity.

3. Data Collection

The following data is assumed to be from local market averages and operational logs to build a realistic model:

- **Revenue per Unit:** Espresso (\$5), Drip Coffee (\$3), Specialty Tea (\$4), Pastry (\$4), Bottled Drink (\$3).
- **Cost per Unit:** Espresso (\$1.00), Drip (\$0.50), Tea (\$0.80), Pastry (\$1.20), Bottled (\$0.50).
- **Labor Requirements:** Espresso (4 min), Tea (2 min), Drip/Pastry (1 min), Bottled (0.5 min).
- **Ingredient Constraints:** Milk usage is critical, with Espresso using 10oz and Tea using 5oz.

4. Variables Definition

To represent the five required product categories, we define the following variables:

- **E:** Total number of Espresso-based drinks produced daily.
- **D:** Total number of Drip coffees brewed daily.
- **T:** Total number of specialty Teas prepared daily.
- **P:** Total number of Pastries stocked daily.
- **B:** Total number of Bottled beverages sold daily.

5. Assumptions & Observations

To ensure the mathematical validity of the model, the following logical assumptions are made:

- **Market Equilibrium:** It is assumed that every item produced is sold, meaning there is no end-of-day surplus.
- **Linear Scaling:** Ingredient costs and preparation times scale linearly (e.g., two coffees take exactly twice as much time as one).
- **Inventory Stability:** Ingredient prices and labor rates are fixed for the duration of the operational day.
- **Total Operational Capacity** Based on historical data and storage limits, the owner identifies a peak daily capacity of **400 total units**. This constraint ensures the shop is fully stocked across all five categories to maximize resource utilization and meet diverse market demand.

6. The System of Equations

Here is a logical model based on common coffee shop data (Revenue, Cost, Labor, and Inventory).

- **Eq 1: Total Revenue Goal**

Assuming prices of \$5, \$3, \$4, \$4, and \$3 respectively:

$$5E + 3D + 4T + 4P + 3B = 1420$$

- **Eq 2: Ingredient Budget**

Assuming the cost to make each item is \$1.00, \$0.50, \$0.80, \$1.20, and \$0.50:

$$1E + 0.5D + 0.8T + 1.2P + 0.5B = 294$$

- **Eq 3: Labor Time (in minutes)**

Total staff time available. Preparation times: 4 min, 1 min, 2 min, 1 min, 0.5 min:

$$4E + 1D + 2T + 1P + 0.5B = 520$$

- **Eq 4: Dairy/Milk Usage (in ounces)**

Espresso and Tea use milk, while Drip and Bottle do not. Total milk supply: 700 oz:

$$10E + 0D + 5T + 0P + 0B = 700$$

- **Eq 5: Daily Unit Capacity**

The owner wants to optimize the shop's operations by producing a total of exactly **400 units** across all product categories each day to match the maximum storage and service capacity.

$$1E - 1D + 1T + 1P + 1B = 400$$

7. Matrix Form ($AX = B$)

The system is translated into the following matrix representation for computational solving:

$$\begin{bmatrix} 5 & 3 & 4 & 4 & 3 \\ 1 & 0.5 & 0.8 & 1.2 & 0.5 \\ 4 & 1 & 2 & 1 & 0.5 \\ 10 & 0 & 5 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} E \\ D \\ T \\ P \\ B \end{bmatrix} = \begin{bmatrix} 1420 \\ 294 \\ 520 \\ 700 \\ 400 \end{bmatrix}$$

Making it a Augmented Matrix:

$$\text{Aug} = \left[\begin{array}{ccccc|c} 5 & 3 & 4 & 4 & 3 & 1420 \\ 1 & 0.5 & 0.8 & 1.2 & 0.5 & 294 \\ 4 & 1 & 2 & 1 & 0.5 & 520 \\ 10 & 0 & 5 & 0 & 0 & 700 \\ 1 & 1 & 1 & 1 & 1 & 400 \end{array} \right]$$

Now we are going to apply row operations to convert it to row Reduced Row Echlon Form (RREF) and then by using back - substitution we are to get the values for our variables one by one.

Solving by applying Row echlon operation:

Swap R1 and R5 to move the leading 1 to the top first position.

$$R1 \leftrightarrow R5 \quad \left[\begin{array}{ccccc|c} 1 & 1 & 1 & 1 & 1 & 400 \\ 1 & 0.5 & 0.8 & 1.2 & 0.5 & 294 \\ 4 & 1 & 2 & 1 & 0.5 & 520 \\ 10 & 0 & 5 & 0 & 0 & 700 \\ 5 & 3 & 4 & 4 & 3 & 1420 \end{array} \right]$$

Now we use the leading 1 in R1 to create zeros in the rest of Column 1.

- **Operations:**
 - $R2 \rightarrow R2 - 1R1$
 - $R3 \rightarrow R3 - 4R1$
 - $R4 \rightarrow R4 - 10R1$
 - $R5 \rightarrow R5 - 5R1$

After applying these operations matrix becomes:

$$\left[\begin{array}{ccccc|c} 1 & 1 & 1 & 1 & 1 & 400 \\ 0 & -0.5 & -0.2 & 0.2 & -0.5 & -106 \\ 0 & -3 & -2 & -3 & -3.5 & -1080 \\ 0 & -10 & -5 & -10 & -10 & -3300 \\ 0 & -2 & -1 & -1 & -2 & -580 \end{array} \right]$$

Now we multiply R2 by -2 to get a leading 1 in the second row

$$\left[\begin{array}{ccccc|c} 1 & 1 & 1 & 1 & 1 & 400 \\ 0 & 1 & 0.4 & -0.4 & 1 & 212 \\ 0 & -3 & -2 & -3 & -3.5 & -1080 \\ 0 & -10 & -5 & -10 & -10 & -3300 \\ 0 & -2 & -1 & -1 & -2 & -580 \end{array} \right]$$

Now we use the leading 1 in row 2 to make rest of the elements of 2 column 0

- **Operations:**

- $R3 \rightarrow R3 + 3R2$
- $R4 \rightarrow R4 + 10R2$
- $R5 \rightarrow R5 + 2R2$
- $R1 \rightarrow R1 - R2$

After applying these operations matrix becomes:

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0.6 & 1.4 & 0 & 188 \\ 0 & 1 & 0.4 & -0.4 & 1 & 212 \\ 0 & 0 & -0.8 & -4.2 & -0.5 & -444 \\ 0 & 0 & -1 & -14 & 0 & -1180 \\ 0 & 0 & -0.2 & -1.8 & 0 & -156 \end{array} \right]$$

- **Operations:** $R3 \rightarrow R4$, then $R3 \rightarrow R3 \times -1$

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0.6 & 1.4 & 0 & 188 \\ 0 & 1 & 0.4 & -0.4 & 1 & 212 \\ 0 & 0 & 1 & -14 & 0 & 1180 \\ 0 & 0 & -0.8 & -4.2 & -0.5 & -444 \\ 0 & 0 & -0.2 & -1.8 & 0 & -156 \end{array} \right]$$

- **Operations:**

- $R1 \rightarrow R1 - 0.6R3$
- $R2 \rightarrow R2 - 0.4R3$
- $R4 \rightarrow R4 + 0.8R3$
- $R5 \rightarrow R5 - 0.2R3$

After applying these operations matrix becomes:

$$\begin{bmatrix} 1 & 0 & 0 & -7 & 0 & | & -520 \\ 0 & 1 & 0 & -6 & 1 & | & -260 \\ 0 & 0 & 1 & 14 & 0 & | & 1180 \\ 0 & 0 & 0 & 7 & -0.5 & | & 500 \\ 0 & 0 & 0 & 1 & 0 & | & 80 \end{bmatrix}$$

Now we need a leading 1 in the 4th row, So after applying operation:

$$R4 \leftrightarrow R5 \quad \begin{bmatrix} 1 & 0 & 0 & -7 & 0 & | & -520 \\ 0 & 1 & 0 & -6 & 1 & | & -260 \\ 0 & 0 & 1 & 14 & 0 & | & 1180 \\ 0 & 0 & 0 & 1 & 0 & | & 80 \\ 0 & 0 & 0 & 7 & -0.5 & | & 500 \end{bmatrix}$$

Using the leading 1 in R4 to create zeros in all other rows of the 4th column.

- **Operations:**
 - $R1 \rightarrow R1 + 7R4$
 - $R2 \rightarrow R2 + 6R4$
 - $R3 \rightarrow R3 - 14R4$
 - $R5 \rightarrow R5 - 7R4$

After applying these operations matrix becomes:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & | & 40 \\ 0 & 1 & 0 & 0 & 1 & | & 220 \\ 0 & 0 & 1 & 0 & 0 & | & 60 \\ 0 & 0 & 0 & 1 & 0 & | & 80 \\ 0 & 0 & 0 & 0 & -0.5 & | & -60 \end{bmatrix}$$

Now we need a leading 1 in the 5th row, So after applying operation:

$$R5 \rightarrow R5 * (-2) \quad \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & | & 40 \\ 0 & 1 & 0 & 0 & 1 & | & 220 \\ 0 & 0 & 1 & 0 & 0 & | & 60 \\ 0 & 0 & 0 & 1 & 0 & | & 80 \\ 0 & 0 & 0 & 0 & 1 & | & 120 \end{bmatrix}$$

Using the leading 1 in R5 to create zeros in all other rows of the 5th column.

R2 → R2 - R5

1	0	0	0	0		40
0	1	0	0	0		100
0	0	1	0	0		60
0	0	0	1	0		80
0	0	0	0	1		120

The Final Decision Plan:

- **Espresso (E):** 40 units
- **Drip Coffee (D):** 100 units
- **Specialty Tea (T):** 60 units
- **Pastries (P):** 80 units
- **Bottled Beverages (B):** 120 units

Key Decision Points

Based on the **Reduced Row Echelon Form (RREF)**, the entrepreneur should execute the following daily inventory plan:

- **Espresso-Based Drinks (E):** Prepare **40 units**. This premium category contributes significantly to revenue while utilizing the most milk and labor per cup.
- **Drip Coffees (D):** Prepare **100 units**. This is the core "high-demand" product, providing a stable foundation for the morning rush.
- **Specialty Teas (T):** Prepare **60 units**. This serves as a balanced mid-tier product that optimizes the remaining milk and labor resources.
- **Assorted Pastries (P):** Prepare **80 units**. These are essential high-margin add-ons that complement the coffee sales without requiring extensive labor.
- **Bottled Beverages (B):** Prepare **120 units**. This is the highest volume category, chosen to maximize the 400-unit capacity with minimal labor and cost requirements.
-

The Real-Life Decision Insight

The core insight of this model is **Resource Optimization**. By solving the system of equations, the owner discovers exactly how to balance expensive, high-labor items (like Espresso) with high-volume, low-effort items (like Bottled Drinks) to hit a specific profit target without wasting a single drop of milk or a minute of staff time.

Strategic Breakdown

- **Balancing the "High-Touch" vs. "High-Volume":** The model shows that while **Espresso (40 units)** is the most profitable per cup, its high labor (4 min) and milk usage (10 oz) mean it cannot be the only product sold. The owner must balance it with **Bottled Drinks (120 units)**, which require almost no labor but help hit the **400-unit capacity** and **\$1,420 revenue** goals.
 - **Inventory Efficiency:** By producing exactly **60 Specialty Teas**, the owner uses the remaining **milk (700 oz total)** perfectly. This prevents "Dead Stock" (wasted milk) which would otherwise cut into the daily procurement budget of **\$294**.
 - **Labor Management:** The decision to sell **100 Drip Coffees** and **80 Pastries** (which only take 1 minute each to prep) ensures the staff is busy but not overwhelmed. The total labor matches the **520-minute** staff shift exactly, preventing both idle time and expensive overtime pay.
 - **Meeting Market Capacity:** Reaching exactly **400 total units** tells the owner that the shop is operating at **100% efficiency**. If the model had resulted in a lower number, it would suggest the shop has "Lazy Capital"—unused space or resources that aren't making money.
-

Computational Implementation

To validate the theoretical results obtained through the matrix inversion method, a computational simulation tool was developed using Python. This 'Daily Grind Simulator' serves as a decision-support system, enabling real-time manipulation of production quantities, operational limits, and unit costs.

This implementation allows us to perform 'what-if' sensitivity analysis, testing the robustness of our optimal production mix against fluctuating resource costs (e.g., milk or labor prices). By automating the calculation of revenue, cost, and gross profit, this software confirms that the matrix solution (40, 100, 60, 80, 120) remains the stable baseline for profit maximization while adhering to all defined operational constraints

```
import numpy as np
```

```
from tabulate import tabulate
```

```
def display_dashboard(q, limits, costs):
```

```
    # Fixed operational constants (Prices are static, Costs are now dynamic)
```

```
    prices = [5, 3, 4, 4, 3]
```

```
    labor = [4, 1, 2, 1, 0.5]
```

```
    milk = [10, 0, 5, 0, 0]
```

```
    names = ["Espresso (E)", "Drip Coffee (D)", "Specialty Tea (T)", "Pastries (P)", "Bottled (B)"]
```

```

# Calculate totals

total_rev, total_cost, total_labor, total_milk, total_units = 0, 0, 0, 0, 0

table_data = []

for i in range(5):

    rev = q[i] * prices[i]

    total_item_cost = q[i] * costs[i]

    profit = rev - total_item_cost


# Adding dynamic columns: Unit Cost, Total Cost, Revenue, Gross Profit
table_data.append([names[i], q[i], f"${costs[i]:.2f}", f"${total_item_cost:.2f}", f"${rev:.2f}", f"${profit:.2f}"])


total_rev += rev

total_cost += total_item_cost

total_labor += (q[i] * labor[i])

total_milk += (q[i] * milk[i])

total_units += q[i]


# Validation logic

c_cap = total_units <= limits['cap']

c_rev = total_rev >= limits['rev']

c_bud = total_cost <= limits['bud']

c_mlk = total_milk <= limits['mlk']

c_lab = total_labor <= limits['lab']


print("\n" + "="*95)

print("    \t\t\tTHE DAILY GRIND: DYNAMIC OPERATIONS SIMULATOR")

print("="*95)

print(tabulate(table_data, headers=["Product", "Qty", "Unit Cost", "Total Cost", "Revenue", "Gross Profit"],
tablefmt="fancy_grid"))

```

```

print(f"\n\t\t\t--- KEY PERFORMANCE INDICATORS ---\n")
print(f"Total Units:  {total_units}/{limits['cap']} " + ("✅" if c_cap else "❌ OVER CAPACITY"))
print(f"Total Revenue: ${total_rev}/${limits['rev']} " + ("✅" if c_rev else "⚠️ BELOW TARGET"))
print(f"Total Budget:  ${total_cost}/${limits['bud']} " + ("✅" if c_bud else "❌ OVER BUDGET"))
print(f"Milk Usage:    {total_milk}oz/{limits['mlk']}oz " + ("✅" if c_mlk else "❌ OUT OF MILK"))
print(f"Staff Labor:    {total_labor}min/{limits['lab']}min " + ("✅" if c_lab else "❌ OVERTIME"))
print("="*95)

```

```
def main():
```

```
    quantities = [40, 100, 60, 80, 120]
```

```
    limits = {'rev': 1420, 'bud': 294, 'mlk': 700, 'lab': 520, 'cap': 400}
```

```
    costs = [1.0, 0.5, 0.8, 1.2, 0.5] # New dynamic costs
```

```
    while True:
```

```
        display_dashboard(quantities, limits, costs)
```

```
        print("\n1: Adjust Qty\n2: Adjust Limits\n3: Adjust Costs\n4: Reset\n5: Exit\n")
```

```
        choice = input("Enter choice: ")
```

```
        # 1: Adjust Quantities
```

```
        if choice == '1':
```

```
            print("Enter 5 new quantities (e.g., 0, 110, 0, 0, 0):")
```

```
            try:
```

```
                new_q = list(map(int, input("Enter New Values --> ").split(',')))
```

```
                if len(new_q) == 5:
```

```
                    for i in range(5):
```

```
                        if new_q[i] != 0: quantities[i] = new_q[i]
```

```
            except: print("Invalid Input.")
```

```
        # 2: Adjust Limits
```

```
        elif choice == '2':
```

```

print("Enter new limits for: Rev, Bud, Milk, Lab, Cap (e.g., 1500, 0, 0, 0, 0):")
try:
    new_l = list(map(int, input("Enter New Values --> ").split(',')))
    if len(new_l) == 5:
        keys = ['rev', 'bud', 'mlk', 'lab', 'cap']
        for i in range(5):
            if new_l[i] != 0: limits[keys[i]] = new_l[i]
    except: print("Invalid Input.")

# 3: Adjust Costs (New)
elif choice == '3':
    print("Enter 5 new unit costs (e.g., 1.2, 0, 0.9, 0, 0):")
    try:
        new_c = list(map(float, input("-> ").split(',')))
        if len(new_c) == 5:
            for i in range(5):
                if new_c[i] != 0: costs[i] = new_c[i]
    except: print("Invalid Input.")

# 4: Reset
elif choice == '4':
    quantities = [40, 100, 60, 80, 120]
    limits = {'rev': 1420, 'bud': 294, 'mlk': 700, 'lab': 520, 'cap': 400}
    costs = [1.0, 0.5, 0.8, 1.2, 0.5]

# 5: Exit
elif choice == '5': break
if __name__ == "__main__":
    main()

```

Initial state:

```
=====
                        THE DAILY GRIND: DYNAMIC OPERATIONS SIMULATOR
=====

Product      Qty  Unit Cost  Total Cost  Revenue  Gross Profit
-----
Espresso (E)   40   $1.00    $40.00    $200.00   $160.00
Drip Coffee (D) 100   $0.50    $50.00    $300.00   $250.00
Specialty Tea (T) 60   $0.80    $48.00    $240.00   $192.00
Pastries (P)   80   $1.20    $96.00    $320.00   $224.00
Bottled (B)   120   $0.50    $60.00    $360.00   $300.00

--- KEY PERFORMANCE INDICATORS ---

Total Units:   400/400  ✓
Total Revenue: $1420/$1420 ✓
Total Budget:  $294.0/$294 ✓
Milk Usage:    700oz/700oz ✓
Staff Labor:   520.0min/520min ✓

=====

1: Adjust Qty
2: Adjust Limits
3: Adjust Costs
4: Reset
5: Exit

Enter choice: 
```

Changing the items quantity:

Enter choice: 1
Enter 5 new quantities (e.g., 0, 110, 0, 0, 0):
Enter New Values --> 30,90,100,0,0

THE DAILY GRIND: DYNAMIC OPERATIONS SIMULATOR

Product	Qty	Unit Cost	Total Cost	Revenue	Gross Profit
Espresso (E)	30	\$1.00	\$30.00	\$150.00	\$120.00
Drip Coffee (D)	90	\$0.50	\$45.00	\$270.00	\$225.00
Specialty Tea (T)	100	\$0.80	\$80.00	\$400.00	\$320.00
Pastries (P)	80	\$1.20	\$96.00	\$320.00	\$224.00
Bottled (B)	120	\$0.50	\$60.00	\$360.00	\$300.00

--- KEY PERFORMANCE INDICATORS ---

Total Units: 420/400 ✗ OVER CAPACITY
Total Revenue: \$1500/\$1420 ✔
Total Budget: \$311.0/\$294 ✗ OVER BUDGET
Milk Usage: 800oz/700oz ✗ OUT OF MILK
Staff Labor: 550.0min/520min ✗ OVERTIME

- 1: Adjust Qty
- 2: Adjust Limits
- 3: Adjust Costs
- 4: Reset
- 5: Exit

Enter choice:

Changing the limits (requirements):

Enter choice: 2

Enter new limits for: Rev, Bud, Milk, Lab, Cap (e.g., 1500, 0, 0, 0, 0):

Enter New Values --> 1500,300,0,600,500

=====

THE DAILY GRIND: DYNAMIC OPERATIONS SIMULATOR

=====

Product	Qty	Unit Cost	Total Cost	Revenue	Gross Profit
Espresso (E)	30	\$1.00	\$30.00	\$150.00	\$120.00
Drip Coffee (D)	90	\$0.50	\$45.00	\$270.00	\$225.00
Specialty Tea (T)	100	\$0.80	\$80.00	\$400.00	\$320.00
Pastries (P)	80	\$1.20	\$96.00	\$320.00	\$224.00
Bottled (B)	120	\$0.50	\$60.00	\$360.00	\$300.00

--- KEY PERFORMANCE INDICATORS ---

Total Units: 420/500 

Total Revenue: \$1500/\$1500 

Total Budget: \$311.0/\$300  OVER BUDGET

Milk Usage: 800oz/700oz  OUT OF MILK

Staff Labor: 550.0min/600min 

- =====
- 1: Adjust Qty
 - 2: Adjust Limits
 - 3: Adjust Costs
 - 4: Reset
 - 5: Exit

Enter choice:

Changing cost per unit:

Enter choice: 3
Enter 5 new unit costs (e.g., 1.2, 0, 0.9, 0, 0):
-> 2,1,1.2,2,1

THE DAILY GRIND: DYNAMIC OPERATIONS SIMULATOR

Product	Qty	Unit Cost	Total Cost	Revenue	Gross Profit
Espresso (E)	30	\$2.00	\$60.00	\$150.00	\$90.00
Drip Coffee (D)	90	\$1.00	\$90.00	\$270.00	\$180.00
Specialty Tea (T)	100	\$1.20	\$120.00	\$400.00	\$280.00
Pastries (P)	80	\$2.00	\$160.00	\$320.00	\$160.00
Bottled (B)	120	\$1.00	\$120.00	\$360.00	\$240.00

--- KEY PERFORMANCE INDICATORS ---

Total Units: 420/500 
Total Revenue: \$1500/\$1500 
Total Budget: \$550.0/\$300  OVER BUDGET
Milk Usage: 800oz/700oz  OUT OF MILK
Staff Labor: 550.0min/600min 

1: Adjust Qty
2: Adjust Limits
3: Adjust Costs
4: Reset
5: Exit

Enter choice:

Final Business "Takeaway"

"To reach our \$1,420 goal today, we shouldn't just sell what is popular; we must sell the specific mix of **40/100/60/80/120** because it is the only way to satisfy our budget, our staff's time, and our fridge space simultaneously."

Conclusion

This project demonstrates the critical role of Linear Algebra in business operations. By representing the production requirements of 'The Daily Grind' as a 5×5 system of linear equations, we successfully identified the intersection of feasibility and profitability.

While the manual matrix solution provided the precise production quantities required to meet our revenue targets, the development of the Daily Grind Simulator transformed these static numbers into a dynamic business tool. We successfully reconciled theoretical mathematical models with practical, data-driven software development. The simulator not only verifies the accuracy of the manual matrix calculations but also provides a scalable framework for future operational planning. This project illustrates that when rigorous mathematics is combined with accessible software tools, it becomes possible to optimize complex systems and make data-backed management decisions.